

Catcher Framing and Challenge System Project

Ben Troutman

2026-1-1

Part 1

Describes the distribution of catcher framing value in runs per season under the current MLB rules.

```
library(dplyr)
library(tidyr)
library(splines)
library(mgcv)
library(ggplot2)
library(xgboost)
library(Matrix)

# Load MLB Data, prepare for strike probability model
mlb_data <- read.csv("mlb_data_prompt.csv") |>
  mutate(strike = event %in% c("called_strike", "strikeout"),
         z_center = (sz_plate_z - (sz_top - sz_bottom)) /
           (sz_top - sz_bottom) - 0.5,
         count = factor(paste(balls, strikes, sep = "-")),
         BatterSide = factor(BatterSide),
         PitcherThrows = factor(PitcherThrows),
         Runners = runner_on_1 + runner_on_2 + runner_on_3,
         ScoreDiff = abs(away_score - home_score))

# Split batters into left and right handed for GAM
lhh_index <- !is.na(mlb_data$BatterSide) & mlb_data$BatterSide == "Left"
rhh_index <- !is.na(mlb_data$BatterSide) & mlb_data$BatterSide == "Right"

# Predict strike probability
gam_L <- gam(
  strike ~ s(sz_plate_x, z_center, k = 55) + PitcherThrows + count +
    RelSpeed + InducedVertBreak + HorzBreak + Runners + ScoreDiff + outs,
  family = binomial,
  data = subset(mlb_data, BatterSide == "Left"),
  method = "REML")

gam_R <- gam(
  strike ~ s(sz_plate_x, z_center, k = 55) + PitcherThrows + count +
```

```

    RelSpeed + InducedVertBreak + HorzBreak + Runners + ScoreDiff + outs,
    family = binomial,
    data = subset(mlb_data, BatterSide == "Right"),
    method = "REML")

mlb_data$StrikeProb[lhh_index] <- predict(
  gam_L,
  newdata = mlb_data[lhh_index, ],
  type = "response")

mlb_data$StrikeProb[rhh_index] <- predict(
  gam_R,
  newdata = mlb_data[rhh_index, ],
  type = "response")

# Calculate extra strikes and runs
mlb_data <- mlb_data |>
  mutate(Residual = strike - StrikeProb,
         ExtraRuns = Residual * 0.125)

mlb_data_summary <- mlb_data |>
  drop_na(StrikeProb) |>
  group_by(catcher_id) |>
  summarize(n_pitches = n(),
            ExtraStrikes = sum(Residual),
            ExtraStrikesPerPitch = ExtraStrikes / n_pitches,
            ExtraRuns = ExtraStrikes * 0.125,
            RunsPer7500 = ExtraRuns / n_pitches * 7500)

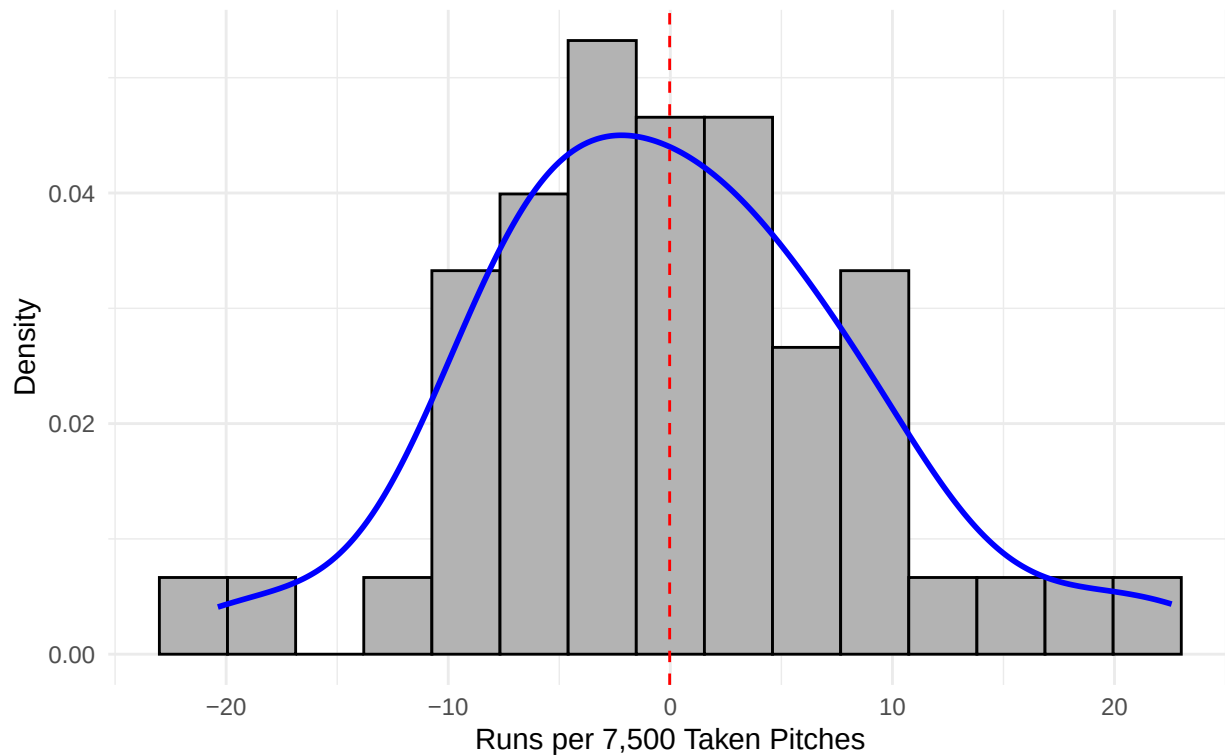
# Function for graphing distributions
plot_histogram <- function(data, title, subtitle) {
  ggplot(data, aes(x = RunsPer7500)) +
    geom_histogram(aes(y = after_stat(density)), bins = 15, fill = "gray70",
                  color = "black") +
    geom_density(linewidth = 1, color = "blue") +
    geom_vline(xintercept = mean(data$RunsPer7500), linetype = "dashed",
              color = "red") +
    labs(title = title, subtitle = subtitle,
         x = "Runs per 7,500 Taken Pitches", y = "Density") +
    theme_minimal()
}

# Plot Residual-based data
plot_histogram(data.frame(RunsPer7500 =
  mlb_data_summary$RunsPer7500[mlb_data_summary$n_pitches >= 3000]),
  "Distribution of Catcher Framing Value (MLB)",
  "Catchers with at least 3,000 taken pitches"
)

```

Distribution of Catcher Framing Value (MLB)

Catchers with at least 3,000 taken pitches



```
# Display summary table
```

```
mlb_data_summary |>
  filter(n_pitches >= 3000) |>
  summarize(Count = n(),
            Mean = mean(RunsPer7500),
            Median = median(RunsPer7500),
            SD = sd(RunsPer7500),
            Min = min(RunsPer7500),
            Max = max(RunsPer7500))
```

```
## # A tibble: 1 x 6
##   Count    Mean Median    SD   Min   Max
##   <int>   <dbl>   <dbl> <dbl> <dbl> <dbl>
## 1     49 -0.0306 -0.380  8.64 -20.4  22.6
```

```
# Load Run Expectancy Data
```

```
# I changed the formatting of the data in Excel to prevent the count from converting to a date
```

```
run_expectancy_data <- read.csv("re288_2023_prompt.csv")
```

```
# Determine "true" strikes using coordinates
```

```
mlb_data <- mlb_data |>
  mutate(TrueStrike = ifelse(sz_plate_x >= -0.83 & sz_plate_x <= 0.83 &
                             sz_plate_z + 0.14 >= sz_bottom & sz_plate_z - 0.14 <= sz_top, 1, 0))
```

```

# Calculate run expectancy changes
mlb_data <- mlb_data |>
  filter(balls <= 3) |> # Remove errant entry with 4 balls
  mutate(Error = as.integer(strike != TrueStrike),
    BallsAfterStrike = ifelse(strikes != 2, balls, 0),
    StrikesAfterStrike = ifelse(strikes != 2, strikes + 1, 0),
    OutsAfterStrike = ifelse(strikes == 2, outs + 1, outs),
    BallsAfterBall = ifelse(balls != 3, balls + 1, 0),
    StrikesAfterBall = ifelse(balls != 3, strikes, 0),
    RunsAfterBall = ifelse(balls == 3 & runner_on_1 & runner_on_2 &
      runner_on_3, 1, 0),
    Runner3AfterBall = ifelse(balls == 3 & runner_on_2 & runner_on_1,
      1, runner_on_3),
    Runner2AfterBall = ifelse(balls == 3 & runner_on_1, 1, runner_on_2),
    Runner1AfterBall = ifelse(balls == 3, 1, runner_on_1)) |>
  left_join(run_expectancy_data, by = c(
    "runner_on_1", "runner_on_2", "runner_on_3", "OutsAfterStrike" = "outs",
    "BallsAfterStrike" = "balls", "StrikesAfterStrike" = "strikes")) |>
  rename(RE_AfterStrike = RE) |>
  left_join(run_expectancy_data, by = c(
    "Runner1AfterBall" = "runner_on_1", "Runner2AfterBall" = "runner_on_2",
    "Runner3AfterBall" = "runner_on_3", "outs",
    "BallsAfterBall" = "balls", "StrikesAfterBall" = "strikes")) |>
  mutate(RE = ifelse(RunsAfterBall, RE + 1, RE),
    RE_AfterStrike = ifelse(OutsAfterStrike == 3, 0, RE_AfterStrike)) |>
  rename(RE_AfterBall = RE) |>
  mutate(DeltaRE = RE_AfterBall - RE_AfterStrike,
    ContextSpecificRuns = Residual * DeltaRE)

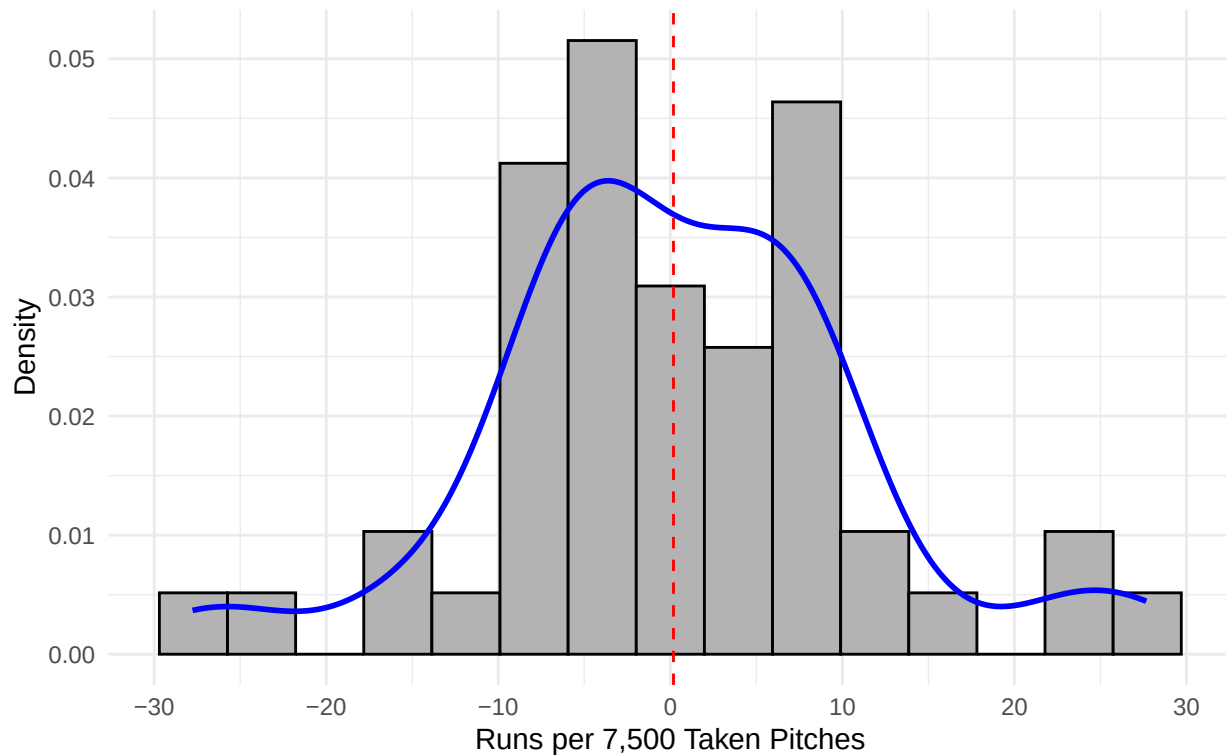
mlb_data_summary_contextualized <- mlb_data |>
  drop_na(StrikeProb) |>
  group_by(catcher_id) |>
  summarize(n_pitches = n(),
    ContextSpecificRuns = sum(ContextSpecificRuns),
    RunsPer7500 = ContextSpecificRuns / n_pitches * 7500)

# Plot Context-Specific Data
plot_histogram(data.frame(RunsPer7500 =
  mlb_data_summary_contextualized$RunsPer7500[
    mlb_data_summary_contextualized$n_pitches >= 3000]),
  "Context-Specific Distribution of Catcher Framing Value (MLB)",
  "Catchers with at least 3,000 taken pitches"
)

```

Context-Specific Distribution of Catcher Framing Value (MLB)

Catchers with at least 3,000 taken pitches



```
mlb_data_summary_contextualized |>
  filter(n_pitches >= 3000) |>
  summarize(Count = n(),
            Mean = mean(RunsPer7500),
            Median = median(RunsPer7500),
            SD = sd(RunsPer7500),
            Min = min(RunsPer7500),
            Max = max(RunsPer7500))
```

```
## # A tibble: 1 x 6
##   Count Mean Median   SD   Min   Max
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     49 0.183 -0.963 10.7 -27.8 27.7
```

Part 2

Assuming a rule change where each team may challenge 2 calls per game, as observed in the minor leagues, describe the distribution of catcher framing value in runs per season under the new rules. Compare this to the results in part 1.

```
# Load MiLB Data
milb_data <- read.csv("milb_data_prompt.csv")
```

```

# Prepare data for strike probability model
milb_data <- milb_data |>
  filter(balls <= 3, strikes <= 2) |> # Remove errant entries
  mutate(strike = event %in% c("called_strike", "strikeout"),
         z_center = (sz_plate_z - (sz_top - sz_bottom)) /
           (sz_top - sz_bottom) - 0.5,
         count = factor(paste(balls, strikes, sep = "-")),
         BatterSide = factor(BatterSide),
         PitcherThrows = factor(PitcherThrows),
         Runners = runner_on_1 + runner_on_2 + runner_on_3,
         ScoreDiff = abs(away_score - home_score),
         challenge_successful = replace_na(challenge_successful, 0),
         CalledStrikeInitially = ifelse((strike & !challenge_successful) |
           (!strike & challenge_successful), TRUE, FALSE))

gam_L_milb <- gam(
  strike ~ s(sz_plate_x, z_center, k = 55) + PitcherThrows + count +
    RelSpeed + InducedVertBreak + HorzBreak + Runners + ScoreDiff + outs,
  family = binomial,
  data = subset(milb_data, BatterSide == "Left"),
  method = "REML")

gam_R_milb <- gam(
  strike ~ s(sz_plate_x, z_center, k = 55) + PitcherThrows + count +
    RelSpeed + InducedVertBreak + HorzBreak + Runners + ScoreDiff + outs,
  family = binomial,
  data = subset(milb_data, BatterSide == "Right"),
  method = "REML")

milb_lhh_index <- !is.na(milb_data$BatterSide) & milb_data$BatterSide == "Left"
milb_rhh_index <- !is.na(milb_data$BatterSide) &
  milb_data$BatterSide == "Right"

milb_data$StrikeProb[milb_lhh_index] <- predict(
  gam_L_milb,
  newdata = milb_data[milb_lhh_index, ],
  type = "response")

milb_data$StrikeProb[milb_rhh_index] <- predict(
  gam_R_milb,
  newdata = milb_data[milb_rhh_index, ],
  type = "response")

milb_data$Residual <- milb_data$CalledStrikeInitially - milb_data$StrikeProb
milb_data$TrueStrike <- ifelse(milb_data$sz_plate_x >= -0.83 &
  milb_data$sz_plate_x <= 0.83 & milb_data$sz_plate_z + 0.14 >=
  milb_data$sz_bottom & milb_data$sz_plate_z - 0.14 <= milb_data$sz_top, 1, 0)

# Calculate RE changes
milb_data <- milb_data |>
  drop_na(z_center, BatterSide, RelSpeed) |>
  mutate(Error = as.integer(CalledStrikeInitially != TrueStrike),
         BallsAfterStrike = ifelse(strikes != 2, balls, 0),

```

```

    StrikesAfterStrike = ifelse(strikes != 2, strikes + 1, 0),
    OutsAfterStrike = ifelse(strikes == 2, outs + 1, outs),
    BallsAfterBall = ifelse(balls != 3, balls + 1, 0),
    StrikesAfterBall = ifelse(balls != 3, strikes, 0),
    RunsAfterBall = ifelse(balls == 3 & runner_on_1 & runner_on_2 &
                           runner_on_3, 1, 0),
    Runner3AfterBall = ifelse(balls == 3 & runner_on_2 & runner_on_1,
                              1, runner_on_3),
    Runner2AfterBall = ifelse(balls == 3 & runner_on_1, 1, runner_on_2),
    Runner1AfterBall = ifelse(balls == 3, 1, runner_on_1)) |>
left_join(run_expectancy_data, by = c(
  "runner_on_1", "runner_on_2", "runner_on_3", "OutsAfterStrike" = "outs",
  "BallsAfterStrike" = "balls", "StrikesAfterStrike" = "strikes")) |>
rename(RE_AfterStrike = RE) |>
left_join(run_expectancy_data, by = c(
  "Runner1AfterBall" = "runner_on_1", "Runner2AfterBall" = "runner_on_2",
  "Runner3AfterBall" = "runner_on_3", "outs",
  "BallsAfterBall" = "balls", "StrikesAfterBall" = "strikes")) |>
mutate(RE = ifelse(RunsAfterBall, RE + 1, RE),
       RE_AfterStrike = ifelse(OutsAfterStrike == 3, 0, RE_AfterStrike)) |>
rename(RE_AfterBall = RE) |>
mutate(DeltaRE = RE_AfterBall - RE_AfterStrike)

# Calculate number of challenges left
milb_data <- milb_data |>
  group_by(game_pk) |>
  fill(challenge_teamid, .direction = "down") |>
  fill(challenge_teamid, .direction = "up") |>
  mutate(FailedChallenge =
    ifelse(challenged == 1 & challenge_successful == 0, 1, 0),
    ChallengesUsedAfter = lag(cumsum(FailedChallenge), default = 0),
    ChallengesLeft = pmax(2 - ChallengesUsedAfter, 0)) |>
  ungroup()

# There are several instances in the data set where teams shouldn't be able to challenge, but still use

# Challenge probability model
data_filtered <- milb_data |> filter(ChallengesLeft > 0)
features <- data_filtered |>
  select(sz_plate_x, z_center, RelSpeed, InducedVertBreak, HorzBreak,
         Runners, ScoreDiff, outs, inning, ChallengesLeft, DeltaRE,
         PitcherThrows, count, TrueStrike, CalledStrikeInitially)
features <- features |>
  mutate(PitcherThrows = as.numeric(factor(PitcherThrows)),
         count = as.numeric(factor(count)),
         across(everything(), as.numeric))
label <- data_filtered$challenged
dtrain <- xgb.DMatrix(data = as.matrix(features), label = label)
params <- list(objective = "binary:logistic", eval_metric = "auc",
              max_depth = 6, eta = 0.1, subsample = 0.8,
              colsample_bytree = 0.8)
xgb_model <- xgb.train(params = params, data = dtrain, nrounds = 200,

```

```

verbose = 1)

# Predict MiLB challenge probability (used for determining challengable opportunities in MLB)
features <- as.matrix(features)
preds <- as.numeric(predict(xgb_model, features))
mlb_data$ChallengeProb[mlb_data$ChallengesLeft > 0] <- preds
mlb_data[mlb_data$ChallengesLeft == 0, ]$ChallengeProb <- 0

# Challenge success model
data_success <- mlb_data |> filter(challenged == 1)
features <- data_success |>
  select(sz_plate_x, z_center, RelSpeed, InducedVertBreak, HorzBreak,
         Runners, ScoreDiff, outs, inning, Error, count, PitcherThrows,
         CalledStrikeInitially, TrueStrike) |>
  mutate(PitcherThrows = as.numeric(factor(PitcherThrows)),
         count = as.numeric(factor(count)))
label <- data_success$challenge_successful
dtrain <- xgb.DMatrix(as.matrix(features), label = label)
params <- list(objective = "binary:logistic", eval_metric = "auc",
              max_depth = 6, eta = 0.1, subsample = 0.8,
              colsample_bytree = 0.8)
xgb_success <- xgb.train(params, data = dtrain, nrounds = 200)

# Created a random sample of challenges left, as there is currently no value for it. Over a large sample
set.seed(1)
mlb_data$ChallengesLeft <- sample(
  mlb_data$ChallengesLeft,
  size = nrow(mlb_data),
  replace = TRUE)

# Predict challenge use for MLB using AAA data
features <- mlb_data |>
  filter(ChallengesLeft > 0) |>
  select(sz_plate_x, z_center, RelSpeed, InducedVertBreak, HorzBreak, Runners,
         ScoreDiff, outs, inning, ChallengesLeft, DeltaRE,
         PitcherThrows, count, TrueStrike, CalledStrikeInitially = strike)
features <- features |>
  mutate(PitcherThrows = as.numeric(factor(PitcherThrows)),
         count = as.numeric(factor(count)))

mlb_data$ChallengeProb[mlb_data$ChallengesLeft > 0] <-
  as.numeric(predict(xgb_model, as.matrix(features)))

mlb_data$ChallengeProb[mlb_data$ChallengesLeft == 0] <- 0

# Predict challenge probability, limiting to realistic situations
mlb_data <- mlb_data |>
  mutate(Challengable =
    ChallengeProb > quantile(mlb_data$ChallengeProb, 0.9))

# Predict challenge effectiveness
features_all <- mlb_data |>

```

```

select(sz_plate_x, z_center, RelSpeed, InducedVertBreak, HorzBreak,
       Runners, ScoreDiff, outs, inning, Error, count, PitcherThrows,
       CalledStrikeInitially = strike, TrueStrike) |>
mutate(PitcherThrows = as.numeric(factor(PitcherThrows)),
       count = as.numeric(factor(count)))

mlb_data$ChallengeSuccessProb <- 0

mlb_data$ChallengeSuccessProb[mlb_data$Challengeable] <-
  predict(xgb_success,
         as.matrix(features_all[mlb_data$Challengeable, ]))

# Summarize data
mlb_data <- mlb_data |>
  mutate(OverturnProb = ChallengeSuccessProb * ChallengeProb,
         AdjustedRuns = ExtraRuns * (1 - OverturnProb),
         RunChange = ExtraRuns - AdjustedRuns)

mlb_data_contextualized <- mlb_data |>
  filter(!is.na(ContextSpecificRuns)) |>
  mutate(OverturnProb = ChallengeSuccessProb * ChallengeProb,
         AdjustedRuns = ContextSpecificRuns * (1 - OverturnProb),
         RunChange = ContextSpecificRuns - AdjustedRuns)

adjusted_mlb_data_summary <- mlb_data |>
  filter(!is.na(AdjustedRuns)) |>
  group_by(catcher_id) |>
  summarize(n_pitches = n(),
            AdjustedRuns = sum(AdjustedRuns),
            RunsPer7500 = AdjustedRuns / n_pitches * 7500,
            RunsWithoutChallenges = sum(ExtraRuns),
            WithoutChallengesPer7500 = RunsWithoutChallenges /
              n_pitches * 7500,
            RunChange = sum(RunChange),
            RunChangePer7500 = RunsPer7500 - WithoutChallengesPer7500)

adjusted_mlb_data_summary_contextualized <- mlb_data_contextualized |>
  group_by(catcher_id) |>
  summarize(n_pitches = n(),
            AdjustedRuns = sum(AdjustedRuns),
            RunsPer7500 = AdjustedRuns / n_pitches * 7500,
            RunsWithoutChallenges = sum(ContextSpecificRuns),
            WithoutChallengesPer7500 = RunsWithoutChallenges /
              n_pitches * 7500,
            RunChange = sum(RunChange),
            RunChangePer7500 = RunsPer7500 - WithoutChallengesPer7500)

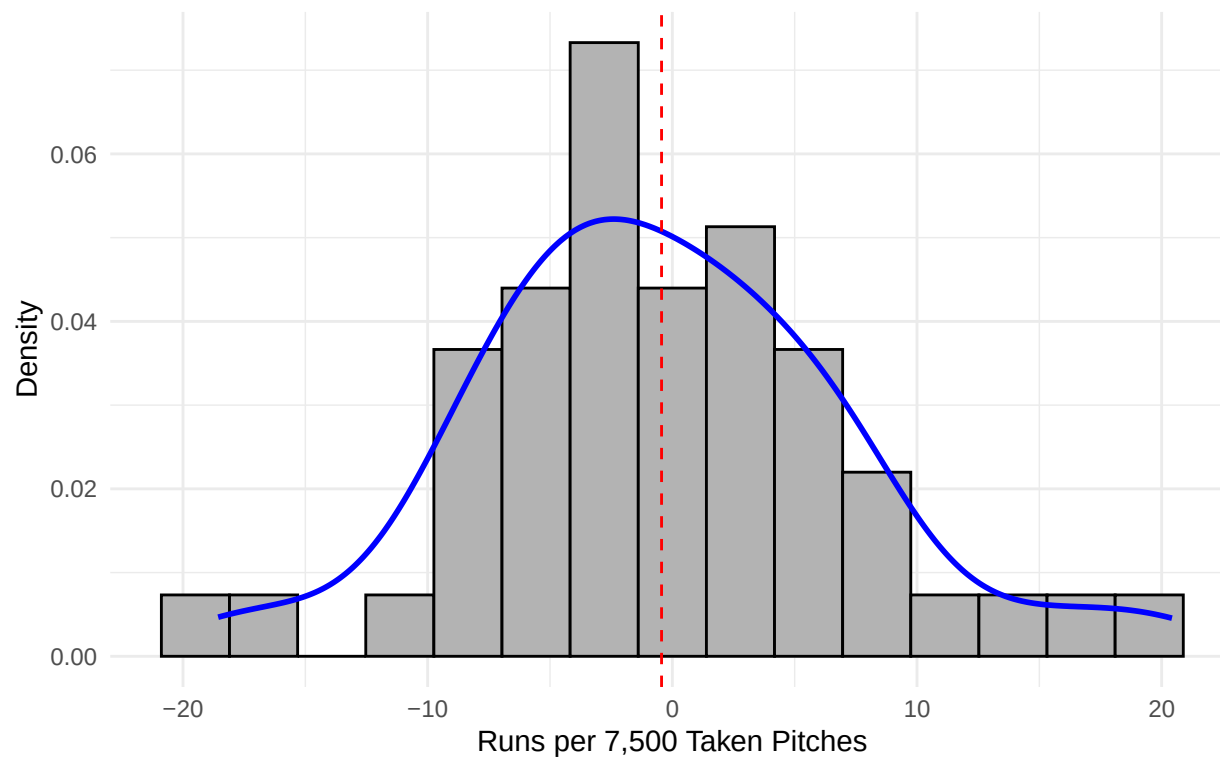
# Plot Challenge-Adjusted Redidual-Based Data
plot_histogram(data.frame(RunsPer7500 =
  adjusted_mlb_data_summary$RunsPer7500[
    adjusted_mlb_data_summary$n_pitches >= 3000]),
  "Challenge-Adjusted Distribution of Catcher Framing (MLB)",
  "Catchters with at least 3,000 taken pitches")

```

)

Challenge-Adjusted Distribution of Catcher Framing (MLB)

Catchers with at least 3,000 taken pitches

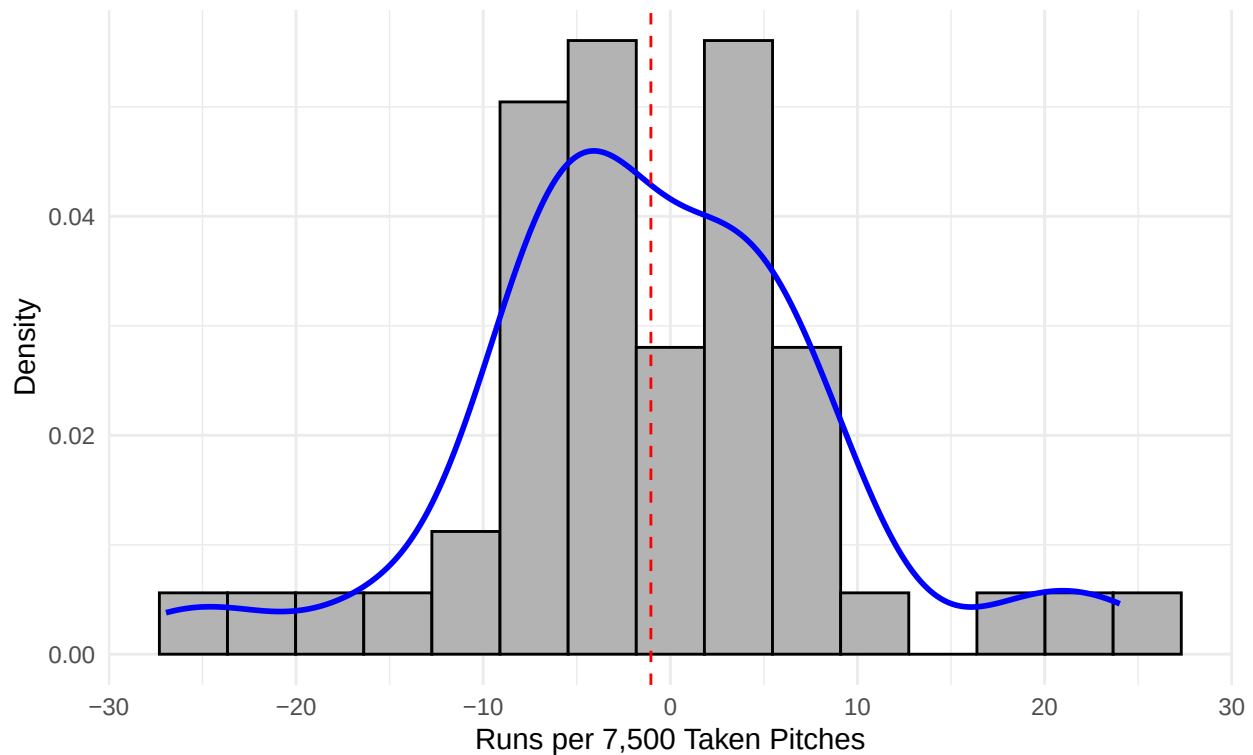


```
adjusted_mlb_data_summary |>
  filter(n_pitches >= 3000) |>
  summarize(Count = n(),
            Mean = mean(RunsPer7500),
            Median = median(RunsPer7500),
            SD = sd(RunsPer7500),
            Min = min(RunsPer7500),
            Max = max(RunsPer7500))
```

```
## # A tibble: 1 x 6
##   Count   Mean Median    SD   Min   Max
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     49 -0.441 -0.725  7.67 -18.6  20.4
```

```
# Plot Challenge-Adjusted Context-Specific Data
plot_histogram(data.frame(RunsPer7500 =
  adjusted_mlb_data_summary_contextualized$RunsPer7500[
    adjusted_mlb_data_summary_contextualized$n_pitches >= 3000]),
  "Context-Specific Challenge-Adjusted Distribution of Framing Value (MLB)",
  "Catchers with at least 3,000 taken pitches"
)
```

Context-Specific Challenge-Adjusted Distribution of Framing Value (MLB) Catchers with at least 3,000 taken pitches



```
adjusted_mlb_data_summary_contextualized |>
  filter(n_pitches >= 3000) |>
  summarize(Count = n(),
            Mean = mean(RunsPer7500),
            Median = median(RunsPer7500),
            SD = sd(RunsPer7500),
            Min = min(RunsPer7500),
            Max = max(RunsPer7500))
```

```
## # A tibble: 1 x 6
##   Count Mean Median    SD  Min  Max
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1    49 -1.04  -2.72  9.64 -26.9  24.0
```

Part 3

Identify 10 catchers in the MLB data-set that would most benefit from the rule change, and 10 MLB catchers that would be most hurt by the rule change.

```
# Residual-based biggest fallers
adjusted_mlb_data_summary |> arrange(RunChangePer7500) |>
  filter(n_pitches >= 3000) |> head(10) |> select(catcher_id, RunsPer7500,
            WithoutChallengesPer7500, RunChangePer7500)
```

```
## # A tibble: 10 x 4
##   catcher_id RunsPer7500 WithoutChallengesPer7500 RunChangePer7500
##   <int>         <dbl>         <dbl>         <dbl>
## 1         21          6.51           9.37         -2.86
## 2        141          7.77          10.4         -2.64
## 3        150         17.2          19.8         -2.58
## 4        182          9.83          12.2         -2.40
## 5        130         20.4          22.6         -2.15
## 6         74          3.70           5.72         -2.03
## 7         62         13.7          15.5         -1.78
## 8         58          6.40           8.18         -1.78
## 9        167          7.72           9.16         -1.44
## 10       118          4.36           5.79         -1.43
```

Residual-based biggest risers

```
adjusted_mlb_data_summary |> arrange(desc(RunChangePer7500)) |>
  filter(n_pitches >= 3000) |> head(10) |> select(catcher_id, RunsPer7500,
    WithoutChallengesPer7500, RunChangePer7500)
```

```
## # A tibble: 10 x 4
##   catcher_id RunsPer7500 WithoutChallengesPer7500 RunChangePer7500
##   <int>         <dbl>         <dbl>         <dbl>
## 1        128        -18.6         -20.4          1.81
## 2        100        -15.5         -17.1          1.57
## 3        179         -9.08         -10.2          1.16
## 4         38         -6.04          -7.08          1.05
## 5         33         -6.14          -7.06          0.915
## 6        162         -8.66          -9.53          0.875
## 7         32         -2.69          -3.51          0.821
## 8         54         -5.97          -6.74          0.772
## 9         43        -11.8         -12.6          0.762
## 10        44         -3.53          -4.13          0.606
```

Context-specific biggest fallers

```
adjusted_mlb_data_summary_contextualized |> arrange(RunChangePer7500) |>
  filter(n_pitches >= 3000) |> head(10) |> select(catcher_id, RunsPer7500,
    WithoutChallengesPer7500, RunChangePer7500)
```

```
## # A tibble: 10 x 4
##   catcher_id RunsPer7500 WithoutChallengesPer7500 RunChangePer7500
##   <int>         <dbl>         <dbl>         <dbl>
## 1        150         18.3          22.4         -4.09
## 2         21          3.48           7.55         -4.07
## 3         62         21.0          24.7         -3.72
## 4        130         24.0          27.7         -3.65
## 5        141         10.6          14.1         -3.50
## 6         74          5.35           8.25         -2.90
## 7         58          3.23           5.83         -2.60
## 8         40        -0.485           1.75         -2.23
## 9         50          1.26           3.47         -2.21
## 10       182          4.15           6.26         -2.11
```

```
# Context-specific biggest risers
adjusted_mlb_data_summary_contextualized |> arrange(desc(RunChangePer7500)) |>
  filter(n_pitches >= 3000) |> head(10) |> select(catcher_id, RunsPer7500,
    WithoutChallengesPer7500, RunChangePer7500)
```

```
## # A tibble: 10 x 4
##   catcher_id RunsPer7500 WithoutChallengesPer7500 RunChangePer7500
##   <int>      <dbl>          <dbl>          <dbl>
## 1      100     -23.3         -24.9           1.61
## 2      162     -11.0         -12.0           1.06
## 3      128     -26.9         -27.8           0.827
## 4       32      -3.33         -3.78           0.453
## 5       43     -13.9         -14.4           0.450
## 6       38      -7.64         -8.08           0.434
## 7       33      -3.84         -4.03           0.190
## 8       54      -5.85         -5.88           0.0323
## 9      148      -3.15         -3.18           0.0321
## 10      179      -7.19         -7.19          -0.00191
```